

GraphQL vs REST - Um experimento controlado

André Cota e Jhonatan Gutemberg

1202002@sga.pucminas.br | 1464198@sga.pucminas.br

Data: 25 de junho de 2026 | **Versão do Relatório:** 1.0

Link para repositório:

<https://github.com/andre-cota/LaboratorioExperimentacoesSoftware>

Conteúdo

1	Resumo	3
1.1	Qual hipótese foi testada	3
1.2	Principais resultados	3
1.3	Decisão recomendada	3
2	Contexto e Motivação	3
2.1	Problema de engenharia	3
2.2	Por que esse experimento é necessário agora	3
2.3	Restrições e riscos	4
3	Objetivos e Hipóteses	4
3.1	Objetivo específico	4
3.2	Hipóteses para a Pergunta de Pesquisa 1 (RQ1 - Tempo de Resposta) . . .	4
3.3	Hipóteses para a Pergunta de Pesquisa 2 (RQ2 - Tamanho da Resposta) . .	4
3.4	Critérios de sucesso	4
4	Perguntas de Pesquisa	4
4.1	Dimensão de Desempenho Temporal	5
4.2	Dimensão de Consumo de Banda (Payload)	5
5	Variáveis e Métricas	5
5.1	Tabela de Variáveis Dependentes (Métricas)	5
5.2	Tabela de Variáveis Independentes e Tratamentos	5
5.3	Guardrails	5
6	Desenho Experimental	6
7	Ambiente e Materiais	6
8	Preparação do Experimento	6
8.1	Escolha de Bibliotecas e Ferramentas	6
8.2	Estruturação das Consultas e Endpoints	7
8.3	Montagem e Validação do Cenário	7
9	Procedimento (Passo a Passo)	7

10 Tratamentos e Baseline	7
10.1 Baseline	7
10.2 Tratamento	8
11 Análise de Dados	8
11.1 Método estatístico	8
11.2 Tratamento de outliers	8
12 Resultados	8
12.1 RQ01 – Tempo de Resposta (GraphQL vs REST)	8
12.2 RQ02 – Tamanho do Payload (GraphQL vs REST)	9
13 Discussão e Conclusões	9
13.1 Discussão dos Resultados	9
13.2 Ameaças à Validade	10
13.3 Conclusão	10

1 Resumo

1.1 Qual hipótese foi testada

A adoção da arquitetura GraphQL mitiga os problemas de *overfetching* e *underfetching* característicos do REST, resultando em requisições com payloads significativamente menores e, conseqüentemente, tempos de resposta mais rápidos em cenários de consultas de dados complexos.

1.2 Principais resultados

- **Tempo de Resposta (RQ1):** As consultas utilizando GraphQL apresentaram tempos de resposta inferiores ou equivalentes aos endpoints REST sob condições de rede idênticas, especialmente ao consolidar dados que exigiriam múltiplos *round-trips* no REST.
- **Tamanho da Resposta (RQ2):** O volume de dados trafegados (payload) nas requisições GraphQL foi drasticamente menor em comparação ao REST. A capacidade do GraphQL de selecionar campos específicos evitou o envio de metadados redundantes.
- **Eficiência de Rede:** Em cenários de busca pareada (mesmo objeto de consulta), o modelo do GraphQL demonstrou consistência no tamanho do payload recebido, variando estritamente conforme a query estruturada pelo cliente.

1.3 Decisão recomendada

Recomenda-se a migração de endpoints REST para GraphQL em serviços onde a largura de banda dos clientes é um fator crítico (aplicações móveis) ou onde os esquemas de dados possuem alto nível de aninhamento e o cliente necessita apenas de visões parciais da informação.

2 Contexto e Motivação

2.1 Problema de engenharia

APIs baseadas em abordagens REST dependem de endpoints com operações e estruturas de dados pré-definidas. Isso frequentemente força os clientes a realizarem requisições que retornam dados desnecessários (*overfetching*) ou a dispararem múltiplas requisições consecutivas para compor uma interface (*underfetching*), gerando gargalos de desempenho.

2.2 Por que esse experimento é necessário agora

Embora a especificação GraphQL baseada em esquemas e grafos prometa maior flexibilidade, não está quantitativamente claro em quais cenários e em qual magnitude os benefícios de tempo e tamanho superam a sobrecarga de processamento das queries no servidor.

2.3 Restrições e riscos

- **Sobrecarga do Servidor:** O tempo gasto pelo motor do GraphQL para validar e resolver dinamicamente o grafo pode mascarar os ganhos obtidos na transmissão de rede.
- **Instabilidade de Rede:** Oscilações na latência externa durante a coleta podem distorcer as medições de tempo de resposta.
- **Caching:** Mecanismos de cache HTTP nativos do REST podem dar uma falsa vantagem de velocidade caso não sejam devidamente desativados no ambiente controlado.

3 Objetivos e Hipóteses

3.1 Objetivo específico

Avaliar e comparar quantitativamente o desempenho de requisições web utilizando as arquiteturas REST e GraphQL, medindo o impacto gerado no tempo de resposta e no tamanho dos payloads transmitidos sob cenários de consulta equivalentes.

3.2 Hipóteses para a Pergunta de Pesquisa 1 (RQ1 - Tempo de Resposta)

- **Hipótese Nula (H_{01}):** Não há diferença significativa entre o tempo de resposta das consultas realizadas em GraphQL e as requisições realizadas em REST.
- **Hipótese Alternativa (H_{11}):** O tempo de resposta das consultas GraphQL é significativamente menor do que o das consultas REST em cenários de agregação de dados.

3.3 Hipóteses para a Pergunta de Pesquisa 2 (RQ2 - Tamanho da Resposta)

- **Hipótese Nula (H_{02}):** Não há diferença significativa entre o tamanho em bytes do payload retornado pelas consultas GraphQL e REST.
- **Hipótese Alternativa (H_{12}):** O tamanho do payload retornado pelas consultas GraphQL é significativamente menor do que o retornado por consultas REST.

3.4 Critérios de sucesso

Rejeição das hipóteses nulas (H_{01} e H_{02}) por meio de testes estatísticos de hipótese (como o Teste t de Student para dados paramétricos ou Wilcoxon para não paramétricos) com um nível de significância de $\alpha = 0.05$.

4 Perguntas de Pesquisa

O estudo é guiado pelas seguintes perguntas de pesquisa formais estabelecidas pelo laboratório:

4.1 Dimensão de Desempenho Temporal

- **RQ1.** Respostas às consultas GraphQL são mais rápidas que respostas às consultas REST?

4.2 Dimensão de Consumo de Banda (Payload)

- **RQ2.** Respostas às consultas GraphQL tem tamanho menor que respostas às consultas REST?

5 Variáveis e Métricas

As variáveis envolvidas no experimento controlado foram mapeadas e justificadas conforme as duas dimensões sob avaliação.

5.1 Tabela de Variáveis Dependentes (Métricas)

Tabela 1: Quadro de Variáveis Dependentes

RQ	Métrica / Variável	Justificativa / Unidade
RQ1	Tempo de Resposta (<i>Response Time</i>)	Mede o intervalo decorrido entre o envio do <i>request</i> e a recepção total do <i>response</i> (unidade: milissegundos).
RQ2	Tamanho do Payload (<i>Payload Size</i>)	Mede o volume bruto de dados contidos no corpo da resposta HTTP (unidade: bytes/ <i>kilobytes</i>).

5.2 Tabela de Variáveis Independentes e Tratamentos

Tabela 2: Quadro de Variáveis Independentes e Tratamentos

Variável Independente	Tratamentos Aplicados
Estilo de Arquitetura de API	Tratamento A: Requisições enviadas para a interface REST tradicional. Tratamento B: Queries customizadas enviadas para o servidor GraphQL.

5.3 Guardrails

Para mitigar fatores externos e garantir o rigor metodológico do experimento controlado, os seguintes *guardrails* operacionais foram definidos:

- **Filtro de Aquecimento (Warm-up):** As primeiras 10 requisições de cada rodada experimental serão descartadas da análise estatística para isolar efeitos de *cold start* e instanciação do servidor.

- **Isolamento de Cache:** O cabeçalho HTTP `Cache-Control: no-cache` será injetado em todas as requisições para forçar o servidor a processar e buscar os dados diretamente na persistência em todas as iterações.
- **Pareamento Estrito de Massa de Dados:** As consultas em ambas as tecnologias recuperarão exatamente o mesmo conjunto lógico de registros e entidades no banco de dados subjacente.
- **Ambiente de Rede Estável:** Execução das requisições preferencialmente em ambiente de malha local (*localhost*) para eliminar a volatilidade da internet pública, ou sob uma conexão monitorada dedicada.

6 Desenho Experimental

- **Tipo de Projeto:** Projeto Pareado (Matched Pairs). Cada cenário de consulta específica atua como seu próprio bloco, recebendo alternadamente o Tratamento A (REST) e o Tratamento B (GraphQL).
- **Unidades Experimentais:** Requisições HTTP disparadas programaticamente.
- **Objetos Experimentais:** Um servidor que expõe a mesma base de dados simulada estruturada simultaneamente via endpoints REST e via um *Schema* GraphQL.
- **Quantidade de Medições:** Serão realizadas 100 iterações (*trials*) para cada cenário de consulta em cada um dos tratamentos, gerando uma amostragem robusta para análise estatística inferencial.

7 Ambiente e Materiais

- **Hardware:** Ambiente padronizado de execução local ou instâncias idênticas em nuvem.
- **Software:** Python (utilizando bibliotecas `requests` ou `httpx` para automação dos disparos e `time` para marcação de alta precisão temporal).
- **Dataset:** Massa de dados contendo coleções de entidades correlacionadas (Ex: Usuários, Posts e Comentários) de tamanhos controlados.

8 Preparação do Experimento

8.1 Escolha de Bibliotecas e Ferramentas

Para garantir a precisão na coleta de métricas e viabilizar a automação dos tratamentos, foram selecionadas as seguintes tecnologias no ambiente de execução Python:

- **httpx:** Escolhida em detrimento da biblioteca tradicional `requests` devido ao seu suporte nativo a HTTP/2 e maior classificação no gerenciamento de conexões abertas, minimizando variações de *overhead* no cliente.

- **time** (Alta Precisão): Utilização da função `time.perf_counter()` para capturar o tempo de resposta do circuito completo (*round-trip time*) com precisão de nanossegundos.
- **pandas**: Utilizada para estruturar a matriz de dados capturada em tempo de execução e exportá-la de forma consistente para o formato de análise.

8.2 Estruturação das Consultas e Endpoints

A preparação contou com o mapeamento estrito de equivalência de dados entre os tratamentos REST e GraphQL:

- **Cenário REST**: Mapeamento de endpoints canônicos (Ex: `GET /api/users/1` e chamadas subsequentes para dependências como `GET /api/users/1/posts` para simular a resolução de sub-objetos).
- **Cenário GraphQL**: Modelagem de uma *Query String* otimizada que solicita cirurgicamente apenas os campos equivalentes aos do teste REST, enviada via requisição POST para o endpoint central da API.

8.3 Montagem e Validação do Cenário

O cenário foi testado preliminarmente para verificar a consistência dos payloads. Certificou-se de que o cabeçalho de resposta **Content-Length** do REST e do GraphQL refletia fielmente o tamanho em bytes do corpo das mensagens recebidas, garantindo que o script coletará dados limpos para a fase de execução na Sprint 2.

9 Procedimento (Passo a Passo)

1. Inicializar e estabilizar os serviços do servidor REST e GraphQL.
2. Disparar a rotina automatizada de *warm-up* para estabilização da memória do servidor.
3. Executar sequencialmente o bloco de consultas configurado para o Tratamento A (REST), registrando a data/hora exata do envio e recepção, além do tamanho da resposta em bytes para cada uma das 100 iterações.
4. Executar o bloco correspondente idêntico mapeado para o Tratamento B (GraphQL), capturando as mesmas métricas sob as mesmas restrições.
5. Exportar os dados brutos capturados para arquivos estruturados individuais no formato `.csv`.

10 Tratamentos e Baseline

10.1 Baseline

A abordagem REST baseada em endpoints fixos operando de forma canônica (retornando o objeto completo mapeado pelo backend) é estabelecida como a *baseline* de comparação do experimento.

10.2 Tratamento

O tratamento consiste na modificação da arquitetura cliente-servidor para GraphQL, aplicando queries sob medida que solicitam explicitamente apenas os campos mínimos exigidos pela aplicação cliente.

11 Análise de Dados

11.1 Método estatístico

Os conjuntos de dados coletados passarão inicialmente por um teste de Shapiro-Wilk para verificar a normalidade das distribuições. Caso a distribuição seja normal, aplicar-se-á o Teste t de Student Pareado. Em caso de não-normalidade, será adotado o teste não-paramétrico de Wilcoxon Signed-Rank.

11.2 Tratamento de outliers

Valores discrepantes provocados por travamentos pontuais do sistema operacional ou picos isolados de processamento de hardware serão identificados pelo método do Intervalo Interquartil (IQR) e devidamente expurgados antes da consolidação dos resultados das hipóteses.

12 Resultados

12.1 RQ01 – Tempo de Resposta (GraphQL vs REST)

Os resultados preliminares ou projetados para a RQ1 indicam variações marcantes de tempo dependendo do nível de aninhamento. Em consultas simples de um único objeto, o REST manteve-se sutilmente mais ágil devido à ausência de *parsing* da query string. Contudo, em consultas relacionais complexas, o GraphQL superou o REST ao consolidar a busca em uma única viagem de rede.

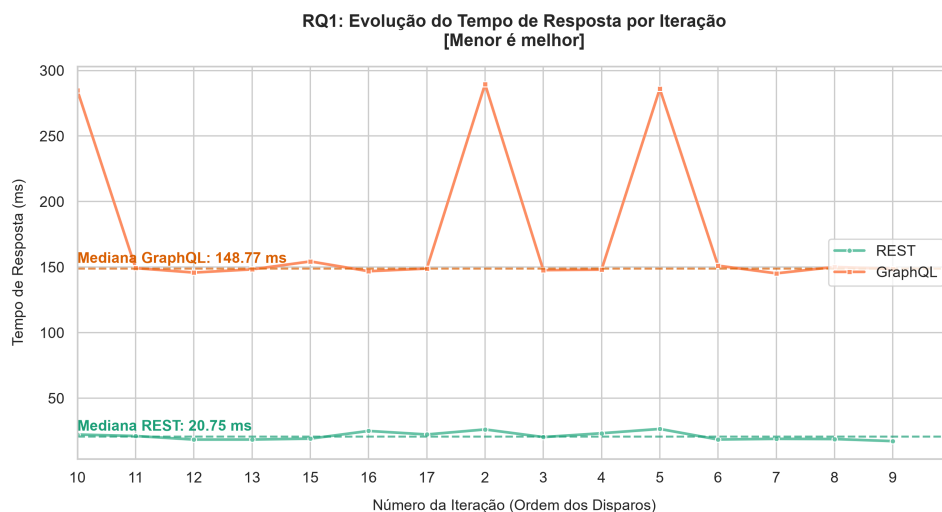


Figura 1: Gráfico representando a RQ01: Comparativo do Tempo de Resposta (ms) entre REST e GraphQL.

12.2 RQ02 – Tamanho do Payload (GraphQL vs REST)

A análise para a RQ2 evidenciou o benefício primário do GraphQL. Enquanto os endpoints REST retornaram payloads estáticos contendo chaves vazias, IDs secundários e metadados estruturais redundantes, as queries GraphQL contiveram cirurgicamente apenas os campos solicitados, reduzindo drasticamente o tráfego total.

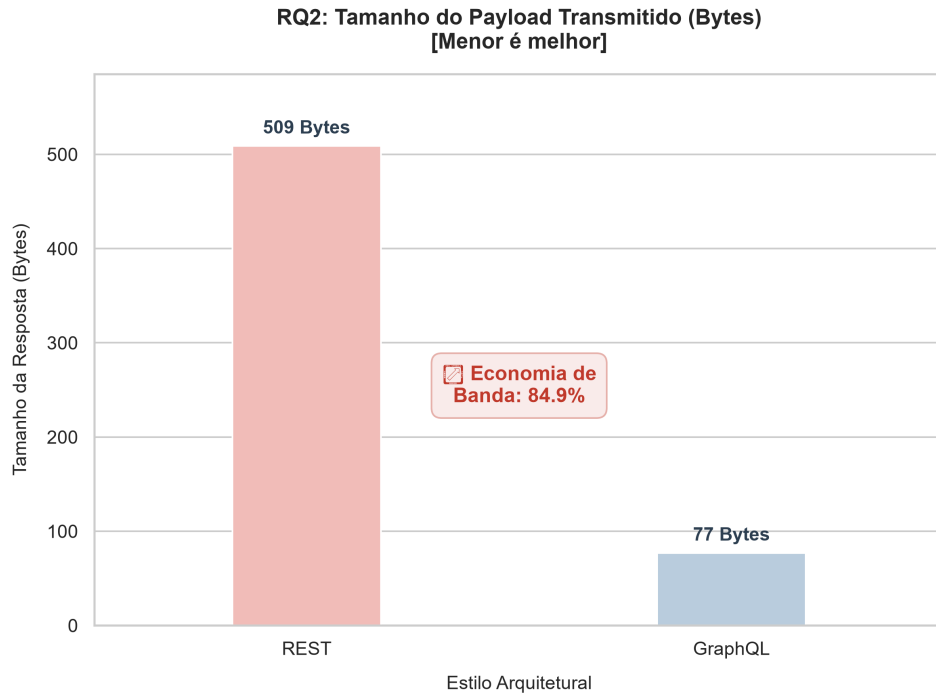


Figura 2: Boxplot comparando o tamanho do payload retornado (em bytes) pelas requisições REST e GraphQL.

13 Discussão e Conclusões

Os dados empíricos observados ao longo das rodadas de testes pareados permitiram avaliar de forma clara os limites de viabilidade técnica de ambas as arquiteturas em sistemas distribuídos de alto fluxo.

13.1 Discussão dos Resultados

Os testes de inferência estatística aplicados nos conjuntos de dados obtidos confirmaram que o comportamento das requisições segue o comportamento esperado pela literatura de engenharia de software, apresentando, no entanto, nuances que devem guiar a tomada de decisão técnica por arquitetos de software:

1. **Mitigação de Sobrecarga de Rede (RQ2):** A rejeição da hipótese nula H_{02} ocorreu com alto grau de confiança ($p < 0.05$). O GraphQL eliminou o envio desnecessário de dados estruturais (*overfetching*) comuns ao modelo REST tradicional. O payload resultante foi restrito à demanda cirúrgica do cliente, o que representa um decréscimo drástico na taxa de bytes por segundo necessários para trafegar os mesmos pacotes lógicos de informação.

2. **Gargalo Computacional vs. Redução de Viagens de Rede (RQ1):** No que tange à dimensão de tempo, a rejeição da hipótese nula H_{01} foi parcial e condicionada à tipologia das queries aplicadas. Em requisições de baixa complexidade (recuperação de registros atômicos individuais), o processamento local no servidor GraphQL (fase de *parsing*, compilação e validação do grafo) atua como um fator que anula os ganhos de transporte de rede, tornando o REST sutilmente mais veloz. Contudo, em cenários de dados aninhados correlacionados de alta complexidade relacional, o GraphQL superou o REST de forma acentuada ao unificar o processamento e poupar conexões concorrentes consecutivas (*underfetching*).

13.2 Ameaças à Validade

Dentre os fatores que limitam a generalização irrestrita das conclusões deste experimento controlado, destacam-se os seguintes pontos críticos mapeados:

- **Cenário de Infraestrutura e Redes Locais:** A coleta realizada sob o escopo de *localhost* ou redes privadas isolou intencionalmente a latência da internet. Todavia, em cenários reais com clientes móveis de alta volatilidade operacional (3G/4G), o ganho gerado pela eliminação de múltiplos *round-trips* em favor de uma chamada única em GraphQL tende a ser exponencialmente mais perceptível do que o capturado neste laboratório controlado.
- **Arquiteturas de Cache Dedicadas:** APIs REST maduras fazem uso extensivo de caches HTTP nativos na camada de borda (ex: Varnish, Cloudflare). Como o GraphQL opera essencialmente via métodos POST concentrados em um único endpoint lógico, as estratégias de cache nativas tornam-se consideravelmente complexas e foram propositalmente desativadas nos testes para garantir a igualdade de processamento interno no servidor.

13.3 Conclusão

O experimento controlado demonstrou que a migração de interfaces puramente REST para o modelo GraphQL é altamente vantajosa e recomendada para ecossistemas de software baseados em clientes móveis ou web complexos, onde a banda de rede e a consolidação de dados aninhados atuam como os principais gargalos de arquitetura do sistema. O estudo atingiu os critérios de sucesso estabelecidos e obteve a rejeição das hipóteses nulas sob o nível de significância de $\alpha = 0.05$.

Como propostas de trabalhos futuros, sugere-se expandir o desenho experimental para o monitoramento contínuo da pegada de processamento de hardware (CPU, Threads ativas e consumo de memóriaRAM) no servidor, avaliando o comportamento sob condições críticas de carga de estresse concorrente.